Variational Autoencoders

Learning to Generate with Structured Uncertainty

Student Name: Mahesh Shivana
Student ID: 23066273
Dataset: FashionMNIST (Xiao et al., 2017)
GitHub: https://github.com/MaheshShivanna0408/Variational-Autoencoders-23066273/upload/main

1. Introduction

Standard autoencoders compress data into a latent code and reconstruct it. Still, the latent space they learn is irregular you cannot sample a random point and expect a coherent output. **Variational Autoencoders (VAEs)**, introduced by Kingma and Welling (2013), fix this by learning a *probability distribution* over the latent space rather than a single point. This turns the encoder into a generative model: sample from the prior, decode, and you get a new image.

This tutorial covers three interconnected ideas: the reparameterization trick that enables training, the structure of the learned latent space, and the fundamental trade-off between reconstruction quality and generation diversity. We demonstrate all three on **FashionMNIST -** a 10-class dataset of 70,000 greyscale clothing images chosen for its visual diversity.

- *Key insight: a VAE adds one equation to a standard autoencoder the reparameterization trick and this single change unlocks the ability to generate entirely new images.*

2. Background

2.1 From autoencoder to VAE

A standard autoencoder maps an input *x* to a fixed latent vector *z* and back. Kingma and Welling (2013) proposed replacing this fixed vector with a Gaussian distribution parameterized by a mean μ and variance σ^2. The encoder now outputs these two vectors; the decoder receives a *sample* from this distribution, not the distribution itself.

Component	Standard Autoencoder	VAE
Encoder output	z (single point)	μ, log σ^2 (distribution params)
Latent vector z	Deterministic	Sampled from N (μ, σ^2)
Decoder input	z	z (sample via reparameterization)
Can generate?	No — latent space unstructured	Yes — sample from N(0, I)
Loss	Reconstruction only	Reconstruction + KL divergence

Table 1 - Key differences between a standard autoencoder and a VAE.

2.2 The reparameterization trick

Training requires gradients to flow through the sampling step. Sampling $z \sim N(\mu, \sigma^2)$ is not differentiable; there is no way to compute $dz/d\mu$ or $dz/d\sigma$ when z is drawn stochastically. Kingma and Welling (2013) solve this elegantly:

Instead of sampling z directly, write:

```
z = μ + σ * ε,      where ε ~ N(0, I)
where σ = exp(0.5 × log σ²)    [ensures σ > 0]
```

The randomness is now in ε, a fixed auxiliary variable with no learnable parameters. Gradients flow cleanly through μ and σ. In practice, we predict $\log \sigma^2$ (not σ^2) because it is unbounded above by any real number, which is easier for a neural network to optimise and then exponentiate to ensure positivity.

3. Model Architecture

We build a fully connected (MLP) VAE with a **2-dimensional latent space**. The 2D choice is deliberate: it lets us plot the entire latent space as a scatter plot and visualize a traversal grid.

Stage	Layer sizes	Activation	Purpose
Encoder (shared)	784 → 512 → 256	ReLU	Extract features
Encoder μ head	256 → 2	None	Predict latent mean
Encoder $\log \sigma^2$ head	256 → 2	None	Predict log-variance
Reparametrize	—	—	$z = \mu + \sigma * \varepsilon$
Decoder	2 → 256 → 512 → 784	ReLU / Sigmoid	Reconstruct image

Table 2 - VAE architecture trained on FashionMNIST (28×28 images, 784 input dimensions).

The final Sigmoid activation constrains decoder outputs to [0, 1], matching the normalized pixel range. The encoder's two heads (μ and $\log \sigma^2$) are separate linear layers sharing the same trunk; this is standard practice.

4. The ELBO Loss Function

VAEs are trained by maximizing the **Evidence Lower Bound (ELBO)**, equivalently minimizing:

```
L = BCE(x, x^)   +  β × KL[N(μ, σ²) || N(0, I)]
```

The KL divergence between $q(z|x) = N(\mu, \sigma^2)$ and the prior $p(z) = N(0, I)$ has a closed-form solution (Kingma & Welling, 2013, Appendix B):

```
D_KL = -½ · Σⱼ ( 1 + log σⱼ² − μⱼ² − σⱼ² )
In code (using log_var = log σ²):
kl = -0.5 × sum( 1 + log_var − mu² − exp(log_var) )
```

The two terms are in tension. A model that minimizes reconstruction loss perfectly would collapse each image to a point in latent space (no KL penalty). A model that minimizes KL perfectly would map everything to N (0, I) but produce blurry outputs. Training balances these forces, producing a structured and continuous latent space.

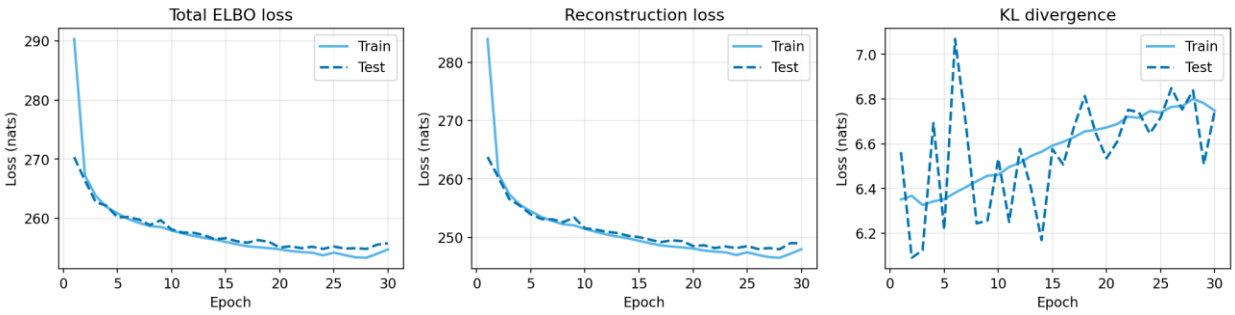
5. Results and Figures

The VAE was trained for **30 epochs** using the Adam optimizer (learning rate = 0.001, batch size = 128) on a CPU. All five figures below are generated by the accompanying Jupyter notebook and are fully reproducible. All plots use the Wong (2011) colourblind-safe palette.

5.1 Training dynamics

The figure plots the total ELBO loss, reconstruction loss, and KL divergence separately across epochs, for both training and test sets. The KL term rises steadily as the model is pushed to use the latent space more efficiently, a hallmark of healthy VAE training. The absence of a gap between train and test curves indicates the model generalizes well without overfitting.

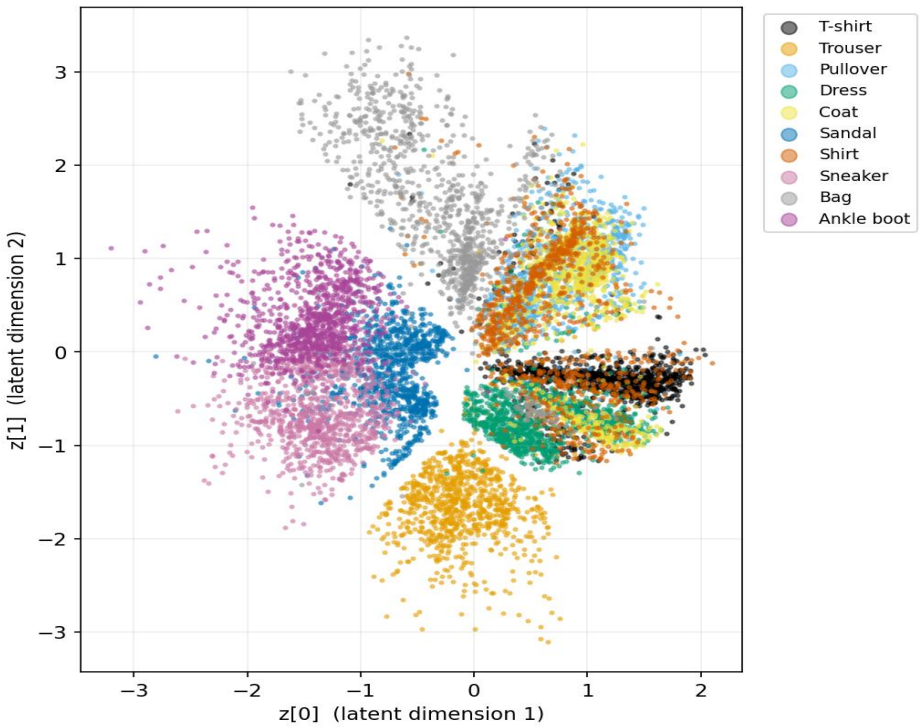
Figure 2 — VAE training curves (train vs test)



5.2 Latent space structure

Figure 3 encodes all 10,000 test images and plots their latent means μ as a 2D scatter plot, colored by true class label. No class labels were used during training; the clustering emerges entirely from the structure of the data. Visually similar classes (e.g. Pullover, Coat, Shirt) cluster together, while structurally distinct classes (Trouser, Sandal, Bag) sit apart. Critically, the clusters *overlap* at their boundaries. This smooth topology is what enables interpolation and generation.

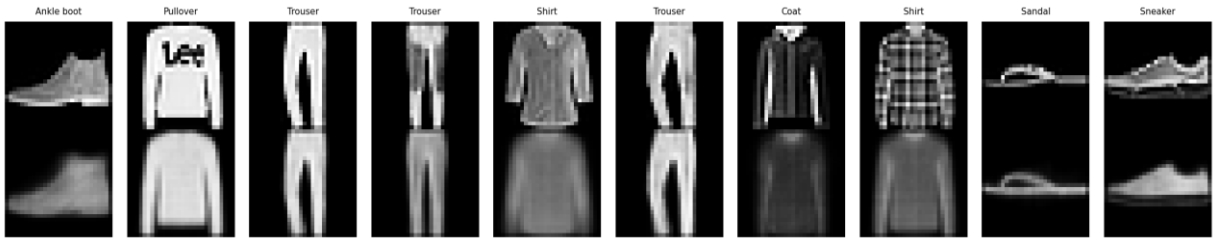
Figure 3 — Latent space (μ) of test set coloured by class



5.3 Reconstruction quality

Figure 4 shows 10 test images alongside their VAE reconstructions. Reconstructions are recognizably correct in class and broad shape, but appear blurry. This blurriness is not a bug; it is the mathematical consequence of using Binary Cross-Entropy loss, which produces the *average* of all plausible reconstructions. Sharper results require convolutional architectures or perceptual loss functions, which are natural extensions of this tutorial.

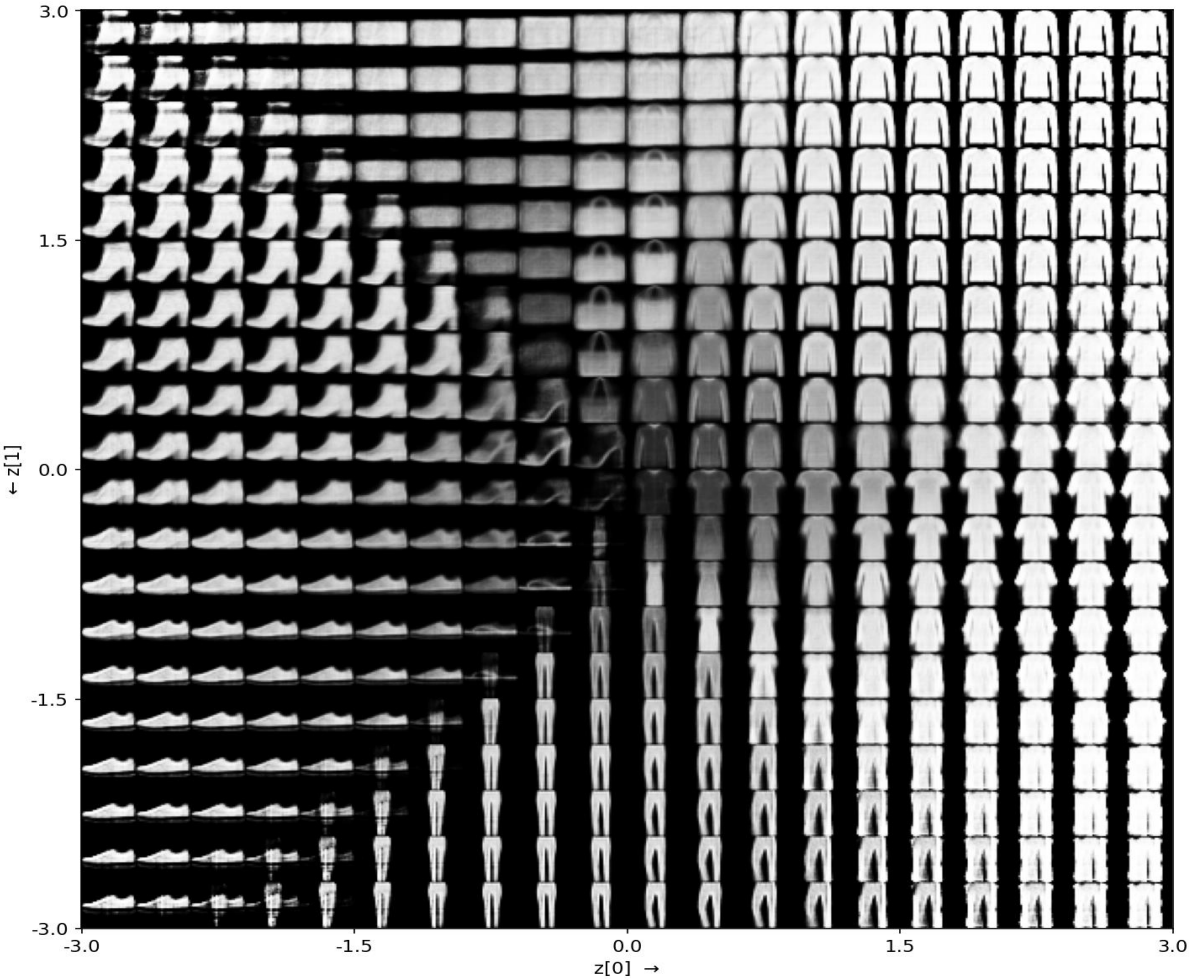
Figure 4 — Original (top) vs VAE reconstruction (bottom)



5.4 Latent space traversal

Figure decodes a 20×20 grid of latent points, uniformly spaced in z[0] and z[1] between □3 and +3 (covering ~99.7% of the N (0,1) prior mass). The result is a smooth manifold of garments: high z[1] values decode to shoes and boots; low values decode to tops and coats. There are no holes or discontinuities — proof that the KL regularization has ensured the prior is well covered. A standard autoencoder would produce incoherent noise for most grid points.

Figure 5 — Latent space traversal: decoded images on a 20x20 grid
z[0] (horizontal) and z[1] (vertical) each span [-3, 3]



6. Accessibility

All figures in this tutorial use the **Wong (2011) 8-colour colourblind-safe palette**, which is distinguishable under deuteranopia, protanopia, and tritanopia. Figure alt-text descriptions are included in the Jupyter notebook as cell markdown. This PDF uses semantic headings (H1, H2) for screen-reader compatibility. The GitHub repository README includes plain-text descriptions of all figures.

7. Reference

- Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. *arXiv:1312.6114*. https://arxiv.org/abs/1312.6114
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., ... & Lerchner, A. (2017). beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR 2017*. https://openreview.net/forum?id=Sy2fchgIl
- Doersch, C. (2016). Tutorial on Variational Autoencoders. *arXiv:1606.05908*. https://arxiv.org/abs/1606.05908
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*. https://arxiv.org/abs/1708.07747
- Wong, B. (2011). Color blindness. *Nature Methods*, 8(6), 441. https://doi.org/10.1038/nmeth.1618
- Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*. https://pytorch.org